
A CLEAR, VISUAL INTRODUCTION

Sorting Algorithms

Bubble Sort • Selection Sort • Insertion Sort

2 3 4 5 8

What Is a Sorting Algorithm?

A sorting algorithm is a precise, step-by-step procedure that takes a collection of items and rearranges them into a defined order — typically ascending or descending — based on a comparison rule.

It doesn't just move items around — it repeats the same rule (compare, then place) until every element sits in its correct position relative to its neighbors.

BEFORE



AFTER



Same elements, one rule applied consistently, a new order.

Why Do We Use Sorting Algorithms?

1

Faster Searching

Sorted data lets us use binary search and similar techniques instead of scanning everything.

2

Efficient Processing

Many algorithms (merging, deduplication, ranking) assume or run best on sorted input.

3

Clarity for Users

Ordered lists, leaderboards, and search results are easier to read and act on.

4

A Learning Foundation

Sorting teaches core ideas — comparisons, loops, complexity — used across computer science.

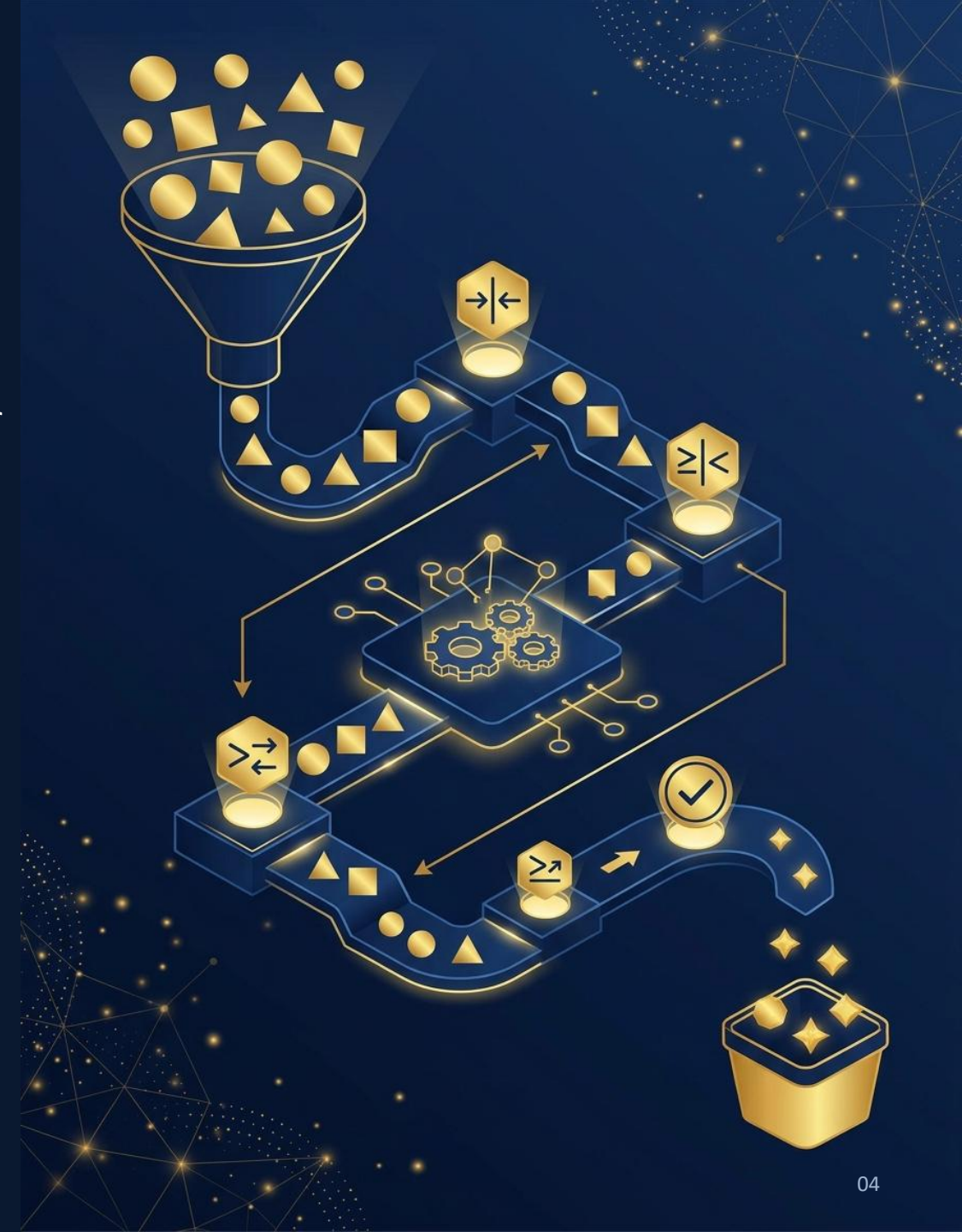
ALGORITHM 01

Bubble Sort — Definition

Bubble Sort repeatedly steps through the list, compares each pair of adjacent elements, and swaps them if they are in the wrong order. Larger values “bubble” toward the end with every pass.

HOW IT WORKS

- 1 Compare each pair of neighboring elements, left to right.
- 2 Swap them if the left one is bigger than the right one.
- 3 Reach the end of the list — the largest value is now in place.
- 4 Repeat for the remaining unsorted portion until no swaps are needed.



Visualizing Bubble Sort

Start
unsorted array



Pass 1
largest bubbles up



Pass 2



Pass 3



Final
fully sorted



 Placed in final position  Still being compared

Selection Sort — Definition

An abstract graphic design on a dark blue background. It features three stylized bar charts. The top-left chart has seven bars of varying heights, some blue and some yellow. A large yellow arrow points from this chart to the top-right chart, which has seven bars of increasing height, all yellow. Below the top-left chart, several yellow and blue lines flow downwards and then curve to the right, pointing towards the bottom-right chart. This bottom-right chart has seven bars of increasing height, all yellow. The background is decorated with faint, glowing yellow and blue geometric shapes, including circles, lines, and triangles, suggesting a network or data structure.

2

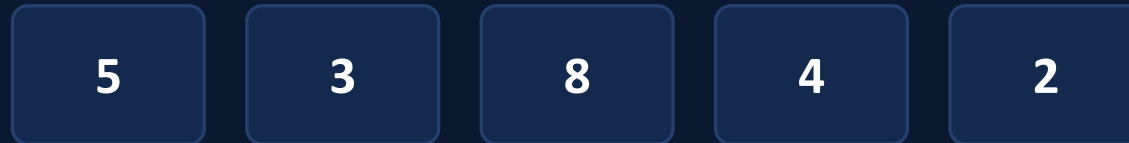
3



Sorting Algorithms

Visualizing Selection Sort

Start
unsorted array



Pass 1
smallest → front



Pass 2



Pass 3



Final
fully sorted



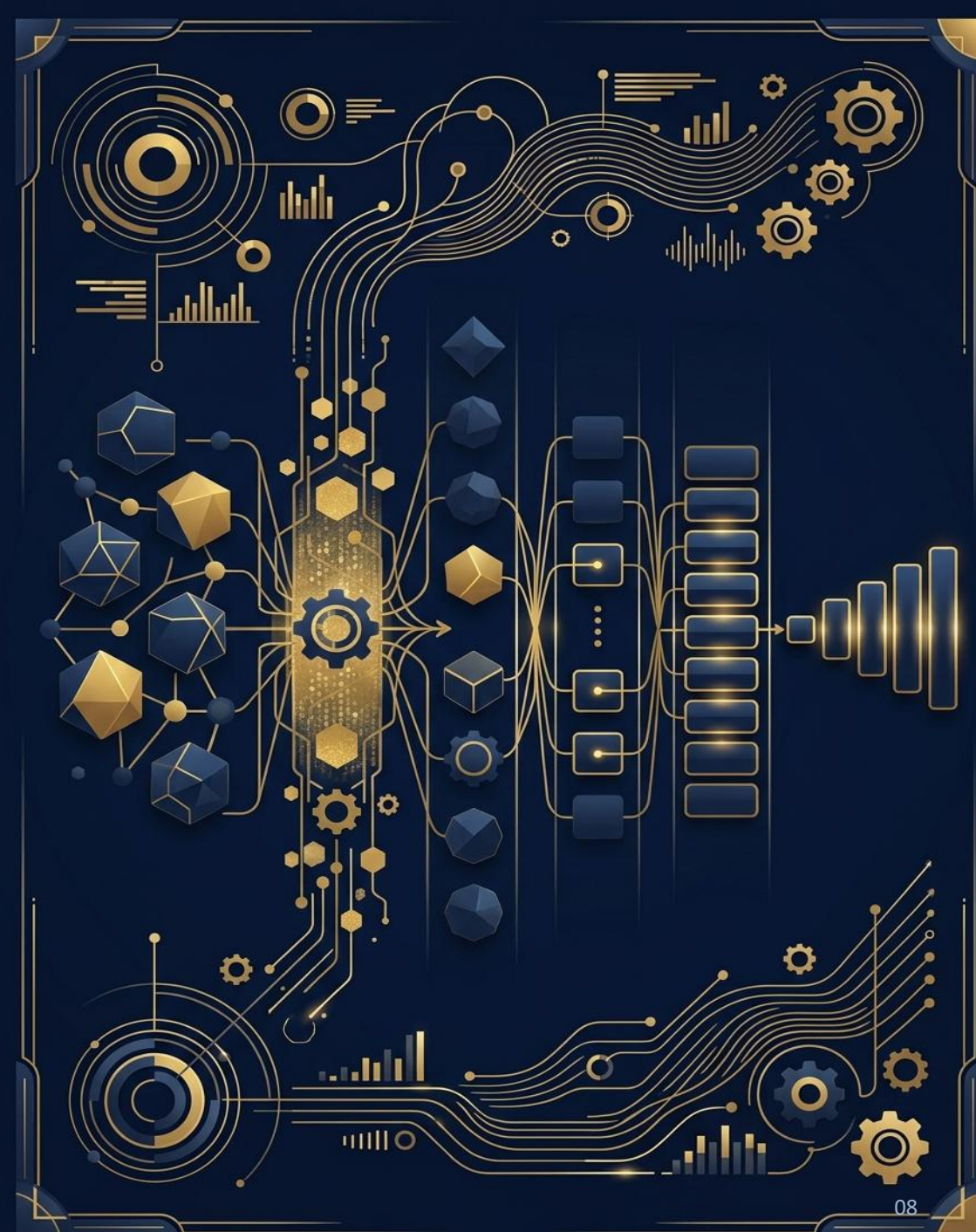
 Sorted portion (front)  Unsorted portion

Insertion Sort — Definition

Insertion Sort builds the sorted list one element at a time. It takes the next unsorted value and inserts it into its correct position among the already-sorted elements, shifting others as needed.

HOW IT WORKS

- 1 Treat the first element as a sorted list of one.
- 2 Take the next element and compare it backward through the sorted part.
- 3 Shift larger elements right to make room.
- 4 Insert the element into its correct slot; repeat for all remaining items.



Visualizing Insertion Sort

Start
first element sorted



Insert 3



Insert 8



Insert 4



Insert 2
fully sorted



 Sorted portion (grows left)  Not yet inserted

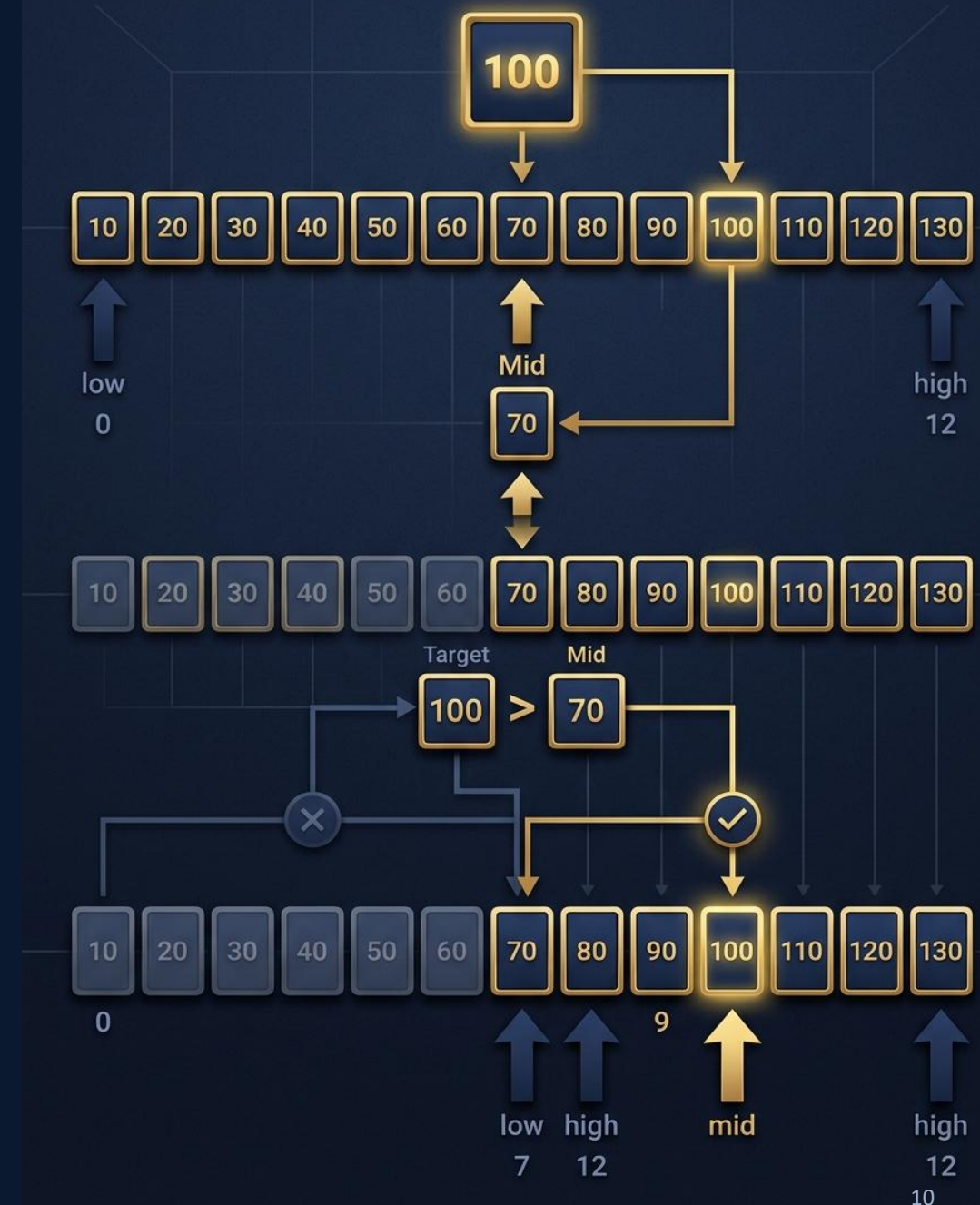
ALGORITHM 04

Binary Search — Definition

Binary Search (or Dichotomous Search) finds a target value in a sorted array by repeatedly splitting the search range in half, discarding the half that cannot contain the target.

HOW IT WORKS

- 1 Start with the full sorted array; set low and high bounds.
- 2 Compare the target to the middle element of the current range.
- 3 If it matches, the search is done — otherwise keep the half that could contain it.
- 4 Repeat on the smaller range until the target is found or the range is empty.



Visualizing Binary Search

Step 1
compare with 9



15 > 9 → search right half

Step 2
compare with 13



15 > 13 → search right half

Step 3
compare with 15



15 = 15 → target found!

Target: 15 • Array is sorted, length 9 • Found in 3 comparisons (vs. up to 9 for a linear scan)

Found

Currently comparing

Still in range

Eliminated

THANK YOU FOR YOUR ATTENTION

Fin

Questions ?

2 3 4 5 8